

```
#####
#
#   COURS : CHAPITRE FINAL – LES 25 DOMAINES FONDAMENTAUX
#   Algèbre · Trigonométrie · Fonctions · Statistiques · Probabilités...
#   Théorie · Démonstrations · Code Python natif
#
#####
#
# CE CHAPITRE COUVRE :
#
# Chaque domaine suit le plan du cours :
# [THÉORIE]      – L'idée, pourquoi ça existe
# [DÉFINITION]   – La règle formelle
# [DÉMONSTRATION] – Preuve pas à pas
# [EXEMPLE]      – Exemple concret
# [CODE PYTHON]  – Programme qui illustre la notion
# [PIÈGE]        – Erreur classique à éviter
#
#####
```

CHAPITRE FINAL – LES 25 DOMAINES FONDAMENTAUX DES MATHÉMATIQUES

PLAN DU CHAPITRE

BLOC A – ALGÈBRE & ARITHMÉTIQUE

1. Algèbre (expressions, équations, polynômes)
2. Systèmes d'équations linéaires
3. Arithmétique modulaire (congruences)
4. Combinatoire (arrangements, combinaisons)

BLOC B – FONCTIONS & ANALYSE

5. Fonctions réelles (généralités)
6. Limites & continuité
7. Dérivation
8. Intégration
9. Suites numériques

BLOC C – TRIGONOMÉTRIE & GÉOMÉTRIE

10. Trigonométrie (cercle trigonométrique)
11. Trigonométrie avancée (formules)
12. Géométrie analytique (droites, cercles)
13. Géométrie vectorielle

BLOC D – ALGÈBRE LINÉAIRE

14. Vecteurs & espaces vectoriels
15. Matrices
16. Déterminants & systèmes

BLOC E – PROBABILITÉS & STATISTIQUES

17. Probabilités (fondamentaux)
18. Probabilités conditionnelles & Bayes
19. Variables aléatoires discrètes
20. Variables aléatoires continues
21. Statistiques descriptives
22. Statistiques inférentielles (bases)

BLOC F – MATHÉMATIQUES AVANCÉES

23. Nombres complexes (applications)
24. Théorie des graphes (bases)
25. Logique & démonstrations (synthèse)

BLOC A – ALGÈBRE & ARITHMÉTIQUE

1. ALGÈBRE – expressions, équations, polynômes

[THÉORIE]

L'algèbre, c'est l'art de manipuler des SYMBOLES à la place de nombres.
On remplace un nombre inconnu par une lettre (x, y, n...) et on manipule des expressions pour trouver ce nombre, ou pour simplifier des formules.
C'est la base de toute la programmation : $x = 3 * y + 1$ est une ligne d'algèbre autant qu'une ligne de Python.

[DÉFINITION]

MONÔME : expression de la forme $a \cdot x^n$ (ex : $3x^2$, $-7x$, 5)
POLYNÔME : somme de monômes $P(x) = a_n x^n + \dots + a_1 x + a_0$
DEGRÉ : exposant du terme dominant (ex : $\deg(3x^3 - x + 2) = 3$)
RACINE de P : valeur r telle que $P(r) = 0$

IDENTITÉS REMARQUABLES :

$(a + b)^2 = a^2 + 2ab + b^2$
 $(a - b)^2 = a^2 - 2ab + b^2$
 $(a + b)(a - b) = a^2 - b^2$
 $(a + b)^3 = a^3 + 3a^2b + 3ab^2 + b^3$

FORMULE DU DISCRIMINANT (degré 2) :

$ax^2 + bx + c = 0$, $\Delta = b^2 - 4ac$
 $\Delta > 0 \rightarrow$ deux racines réelles $x = (-b \pm \sqrt{\Delta}) / 2a$
 $\Delta = 0 \rightarrow$ une racine double $x = -b / 2a$
 $\Delta < 0 \rightarrow$ pas de racine réelle (deux racines complexes)

[DÉMONSTRATION] Formule du discriminant

On cherche x tel que $ax^2 + bx + c = 0$ ($a \neq 0$).

On divise par a : $x^2 + (b/a)x + c/a = 0$

Complétion du carré :

$(x + b/2a)^2 - b^2/4a^2 + c/a = 0$
 $(x + b/2a)^2 = b^2/4a^2 - c/a = (b^2 - 4ac)/4a^2$
 $x + b/2a = \pm \sqrt{(b^2 - 4ac)} / 2a$
 $x = (-b \pm \sqrt{(b^2 - 4ac)}) / 2a \quad \square$

[EXEMPLE]

$2x^2 - 5x + 3 = 0$
 $\Delta = 25 - 24 = 1 > 0$
 $x_1 = (5 + 1)/4 = 3/2 \quad x_2 = (5 - 1)/4 = 1$
Vérification : $2(3/2)^2 - 5(3/2) + 3 = 9/2 - 15/2 + 6/2 = 0 \quad \checkmark$

[CODE PYTHON]

```
import math

def resoudre_degre2(a, b, c):
    """Résout  $ax^2 + bx + c = 0$ """
    delta = b**2 - 4*a*c
    print(f"Équation : {a}x^2 + {b}x + {c} = 0")
    print(f" $\Delta = {b}^2 - 4 \times {a} \times {c} = {delta}$ ")
    if delta > 0:
        x1 = (-b + math.sqrt(delta)) / (2*a)
        x2 = (-b - math.sqrt(delta)) / (2*a)
        print(f"Deux racines :  $x_1 = {x1:.4f}$ ,  $x_2 = {x2:.4f}$ ")
        return x1, x2
    elif delta == 0:
        x = -b / (2*a)
        print(f"Racine double :  $x = {x:.4f}$ ")
        return x, x
    else:
        print(f"Pas de racine réelle ( $\Delta < 0$ )")
        return None

resoudre_degre2(2, -5, 3)    #  $x=1.5$ ,  $x=1$ 
resoudre_degre2(1, -2, 1)   # racine double  $x=1$ 
resoudre_degre2(1, 0, 1)    #  $\Delta < 0$ , pas de racine réelle

# Identités remarquables
def verifier_identites(a, b):
    print(f"\nPour a={a}, b={b} :")
    print(f" $(a+b)^2 = {(a+b)**2}$  vs  $a^2+2ab+b^2 = {a**2+2*a*b+b**2}$ ")
    print(f" $(a-b)^2 = {(a-b)**2}$  vs  $a^2-2ab+b^2 = {a**2-2*a*b+b**2}$ ")
    print(f" $(a+b)(a-b) = {(a+b)*(a-b)}$  vs  $a^2-b^2 = {a**2-b**2}$ ")

verifier_identites(3, 5)
```

[PIÈGE]

Confondre $x^2 = 4$ et $x = 4/2$.

$x^2 = 4 \Rightarrow x = \pm 2$ (deux solutions, pas une seule).

Toujours penser aux racines négatives !

2. SYSTÈMES D'ÉQUATIONS LINÉAIRES

[THÉORIE]

Un système linéaire, c'est plusieurs équations du premier degré avec plusieurs inconnues. Géométriquement, chaque équation est une droite (en 2D) ou un plan (en 3D). La solution est l'intersection.

[DÉFINITION]

Système 2x2 :

$$a_1x + b_1y = c_1$$

$$a_2x + b_2y = c_2$$

Trois cas :

→ Solution unique : les droites se croisent en un point

→ Infinité de solutions : les droites sont confondues

→ Pas de solution : les droites sont parallèles

MÉTHODE DE SUBSTITUTION : exprimer une variable en fonction de l'autre.

MÉTHODE D'ÉLIMINATION (Gauss) : combiner les équations pour éliminer une variable.

[DÉMONSTRATION] Résolution par Gauss de $2x + y = 5$ et $x - y = 1$

$$\text{Eq1 : } 2x + y = 5$$

$$\text{Eq2 : } x - y = 1$$

Additionner Eq1 + Eq2 :

$$(2x + x) + (y - y) = 5 + 1$$

$$3x = 6 \rightarrow x = 2$$

Substituer dans Eq2 :

$$2 - y = 1 \rightarrow y = 1$$

Vérification dans Eq1 : $2(2) + 1 = 5 \checkmark \quad \square$

[CODE PYTHON]

```
import numpy as np

# Résoudre le système Ax = b
A = np.array([[2, 1],
              [1, -1]])
b = np.array([5, 1])

x = np.linalg.solve(A, b)
print(f"Solution : x = {x[0]:.2f}, y = {x[1]:.2f}") # x=2, y=1

# Vérification
residual = A @ x - b
print(f"Vérification (résidu) : {residual}") # [0. 0.]

# Visualisation : les deux droites
import numpy as np

def afficher_systeme(a1, b1, c1, a2, b2, c2):
    """Résout a1x+b1y=c1 et a2x+b2y=c2"""
    A = np.array([[a1, b1], [a2, b2]])
    b_vec = np.array([c1, c2])
    det = np.linalg.det(A)
    if abs(det) < 1e-10:
        print("Pas de solution unique (droites parallèles ou confondues)")
        return
    sol = np.linalg.solve(A, b_vec)
    print(f"Solution unique : x={sol[0]:.4f}, y={sol[1]:.4f}")

afficher_systeme(2, 1, 5, 1, -1, 1) # x=2, y=1
afficher_systeme(2, 4, 6, 1, 2, 3) # infinité (même droite)
```

[PIÈGE]

Un système avec plus d'équations que d'inconnues peut n'avoir AUCUNE solution (sur-déterminé). Toujours vérifier le rang de la matrice.

3. ARITHMÉTIQUE MODULAIRE – les congruences

[THÉORIE]

L'arithmétique modulaire c'est "l'arithmétique de l'horloge" : après 12h on revient à 0. Le module est la valeur à partir de laquelle on repart à 0. C'est la base de la cryptographie, des hashes et des codes correcteurs.

[DÉFINITION]

$a \equiv b \pmod{n} \iff n \mid (a - b) \iff a \bmod n = b \bmod n$

PROPRIÉTÉS :

Si $a \equiv b$ et $c \equiv d \pmod{n}$ alors :

- $a + c \equiv b + d \pmod{n}$
- $a \cdot c \equiv b \cdot d \pmod{n}$

PETIT THÉORÈME DE FERMAT :

Si p premier et $p \nmid a$, alors $a^{p-1} \equiv 1 \pmod{p}$

[DÉMONSTRATION] $2^{10} \bmod 7$ via Fermat

7 est premier, $\text{pgcd}(2,7) = 1$.

Fermat : $2^6 \equiv 1 \pmod{7}$

Donc $2^{10} = 2^6 \cdot 2^4 \equiv 1 \cdot 2^4 = 16 \equiv 2 \pmod{7}$

Vérification : $2^{10} = 1024 = 146 \times 7 + 2 \quad \checkmark \quad \square$

[CODE PYTHON]

```
# Arithmétique modulaire
print(17 % 5)          # 2    (17 ≡ 2 mod 5)
print(pow(2, 10, 7))   # 2    (210 mod 7, efficace même pour grands nombres)

# Petit théorème de Fermat
p = 7
a = 3
print(f"\n{a}^{(p-1)} mod {p} =", pow(a, p-1, p)) # 1 (si p premier)

# Inverse modulaire : a · a-1 ≡ 1 (mod n)
def inverse_mod(a, n):
    """Retourne l'inverse de a modulo n (si existe)"""
    return pow(a, -1, n) # Python 3.8+

print("\nInverse de 3 mod 7 :", inverse_mod(3, 7)) # 5 (car 3×5=15≡1 mod 7)
print("Vérification :", (3 * 5) % 7)               # 1 ✓

# Application : heure de l'horloge
heure_actuelle = 10
heures_ecoulees = 27
heure_finale = (heure_actuelle + heures_ecoulees) % 12
print(f"\nHeure finale : {heure_finale}h") # 1h

# Chiffrement César (rot13)
def chiffrer_cesar(texte, decalage):
    resultat = ""
    for c in texte.upper():
        if c.isalpha():
            resultat += chr((ord(c) - ord('A') + decalage) % 26 + ord('A'))
        else:
            resultat += c
    return resultat

msg = "BONJOUR"
chiffre = chiffrer_cesar(msg, 13)
dechiffre = chiffrer_cesar(chiffre, 13)
print(f"\n{msg} → {chiffre} → {dechiffre}")
```

[PIÈGE]

En Python, l'opérateur % pour les négatifs suit les règles Python,

pas forcément les conventions mathématiques :
-1 % 7 = 6 en Python (correct), mais attention dans d'autres langages !

4. COMBINATOIRE – arrangements et combinaisons

[THÉORIE]

Combien de façons peut-on choisir ou arranger des objets ?
C'est la COMBINATOIRE. Elle sert en probabilités, en algorithmique
et chaque fois qu'on compte des possibilités.

[DÉFINITION]

FACTORIELLE : $n! = n \times (n-1) \times \dots \times 2 \times 1$ ($0! = 1$ par convention)

ARRANGEMENTS (permutations avec ordre, sans remise) :
 $A^*_k = n! / (n-k)! \quad ("k \text{ parmi } n \text{ ordonnés}")$

COMBINAISONS (sans ordre, sans remise) :
 $C(n,k) = n! / (k! \times (n-k)!)$ ("k parmi n")
Aussi noté $\binom{n}{k}$ (coefficient binomial)

TRIANGLE DE PASCAL : $C(n,k) = C(n-1,k-1) + C(n-1,k)$

FORMULE DU BINÔME : $(a+b)^n = \sum_k C(n,k) \cdot a^{n-k} \cdot b^k$

[DÉMONSTRATION] $C(n,k) = C(n-1,k-1) + C(n-1,k)$

On a n objets. On en choisit k .

Fixons un objet O . Deux cas :

O est choisi : il reste à choisir $k-1$ parmi $n-1 \rightarrow C(n-1,k-1)$ façons

O n'est pas choisi : on choisit k parmi $n-1 \rightarrow C(n-1,k)$ façons

Ces cas s'excluent et couvrent tout : $C(n,k) = C(n-1,k-1) + C(n-1,k) \quad \square$

[CODE PYTHON]

```
import math
from itertools import combinations, permutations

# Factorielle
print(math.factorial(5))    # 120

# Arrangements  $A(n,k) = n!/(n-k)!$ 
def arrangements(n, k):
    return math.factorial(n) // math.factorial(n - k)

print("\nA(5,2) =", arrangements(5, 2))    # 20

# Combinaisons  $C(n,k)$ 
def C(n, k):
    return math.comb(n, k)    # Disponible depuis Python 3.8

print("C(5,2) =", C(5, 2))    # 10

# Liste des combinaisons
print("\nCombinaisons de {A,B,C,D} prises 2 à 2 :")
elements = ['A', 'B', 'C', 'D']
for combo in combinations(elements, 2):
    print(" ", combo)

# Triangle de Pascal
def pascal(n):
    triangle = [[1]]
    for i in range(1, n+1):
        ligne = [1]
        for j in range(1, i):
            ligne.append(triangle[i-1][j-1] + triangle[i-1][j])
        ligne.append(1)
        triangle.append(ligne)
    return triangle

print("\nTriangle de Pascal (6 lignes) :")
for ligne in pascal(5):
    print(" ", ligne)
```

```
# Formule du binôme : (a+b)^n
def binome(a, b, n):
    return sum(C(n, k) * a**(n-k) * b**k for k in range(n+1))

print("\n(2+3)^4 =", binome(2, 3, 4), "vs direct :", (2+3)**4) # 625
```

[PIÈGE]

$C(n, k)$ suppose des éléments DISTINCTS. Si les éléments se répètent, la formule change. Et attention : $C(n, 0) = C(n, n) = 1$, jamais 0.

BLOC B – FONCTIONS & ANALYSE

5. FONCTIONS RÉELLES – généralités

[THÉORIE]

Une fonction, c'est une machine : on lui donne un input (x), elle rend un output f(x). Exactement comme une fonction en Python. Mais en maths on s'intéresse à des propriétés globales : est-elle croissante ? bornée ? paire ? périodique ?

[DÉFINITION]

$f : D \rightarrow \mathbb{R}$ associe à chaque $x \in D$ une valeur $f(x) \in \mathbb{R}$
 D est le DOMAINE DE DÉFINITION (ensemble des x valides).

PARITÉ :

f paire : $f(-x) = f(x)$ (symétrie axe 0y) ex : x^2 , $\cos(x)$
 f impaire : $f(-x) = -f(x)$ (symétrie par rapport à 0) ex : x^3 , $\sin(x)$

MONOTONIE :

f croissante : $x_1 < x_2 \Rightarrow f(x_1) < f(x_2)$
 f décroissante : $x_1 < x_2 \Rightarrow f(x_1) > f(x_2)$

PÉRIODICITÉ :

f est T-périodique : $\forall x, f(x + T) = f(x)$

FONCTIONS DE RÉFÉRENCE :

$x \rightarrow$ polynômes (continus partout)
 $1/x \rightarrow$ domaine $\mathbb{R} \setminus \{0\}$
 $\sqrt{x} \rightarrow$ domaine $[0, +\infty[$
 $e^x \rightarrow$ exponentielle, toujours > 0
 $\ln(x) \rightarrow$ domaine $]0, +\infty[$

[DÉMONSTRATION] $\ln(ab) = \ln(a) + \ln(b)$

On pose $f(x) = \ln(ax) - \ln(x)$ pour $x > 0$.
 $f'(x) = a/(ax) - 1/x = 1/x - 1/x = 0$
 f est constante ! Donc $f(x) = f(1) = \ln(a \cdot 1) - \ln(1) = \ln(a) - 0 = \ln(a)$.
 Ainsi $\ln(ax) = \ln(a) + \ln(x)$, i.e. $\ln(ab) = \ln(a) + \ln(b)$ $\square$

[CODE PYTHON]

```
import math

# Propriétés d'une fonction
def est_paire(f, domaine):
    return all(abs(f(x) - f(-x)) < 1e-9 for x in domaine if -x in domaine)

def est_impaire(f, domaine):
    return all(abs(f(x) + f(-x)) < 1e-9 for x in domaine if -x in domaine)

domaine = [x/10 for x in range(-50, 51)] # [-5, ..., 5]

print("x^2 est paire :", est_paire(lambda x: x**2, domaine)) # True
print("x^3 est impaire :", est_impaire(lambda x: x**3, domaine)) # True
print("x^2+x est paire :", est_paire(lambda x: x**2 + x, domaine)) # False

# Domaine de définition (là où la fonction est définie)
def domaine_racine(x_min, x_max, pas=0.1):
    return [x for x in [i*pas for i in range(int(x_min/pas), int(x_max/pas)+1)]
```

```

    if x >= 0] #  $\sqrt{x}$  défini seulement pour  $x \geq 0$ 

# Vérifier  $\ln(ab) = \ln(a) + \ln(b)$ 
a, b = 3.5, 2.7
print(f"\nln({a}×{b}) = {math.log(a*b):.6f}")
print(f"ln({a})+ln({b}) = {math.log(a)+math.log(b):.6f}") # Identiques ✓

```

[PIÈGE]

$\ln(x + y) \neq \ln(x) + \ln(y)$ (erreur très fréquente !)
 C'est $\ln(x \cdot y) = \ln(x) + \ln(y)$ (produit, pas somme).

6. LIMITES & CONTINUITÉ

[THÉORIE]

"Vers quoi tend $f(x)$ quand x s'approche d'une valeur ?"
 C'est la limite. Elle permet d'étudier le comportement d'une fonction
 près d'un point problématique (division par 0, infini...).
 La continuité, c'est "on peut tracer la courbe sans lever le crayon".

[DÉFINITION]

$\lim_{x \rightarrow a} f(x) = L$ signifie :
 Pour tout $\varepsilon > 0$, il existe $\delta > 0$ tel que
 $|x - a| < \delta \implies |f(x) - L| < \varepsilon$

f est CONTINUE en a : $\lim_{x \rightarrow a} f(x) = f(a)$
 (la limite existe ET vaut $f(a)$)

FORMES INDÉTERMINÉES :

$0/0$, ∞/∞ , $0 \times \infty$, $\infty - \infty$, 0^0 , 1^∞ , ∞^0
 → Utiliser L'Hôpital, factorisations, équivalents

RÈGLE DE L'HÔPITAL :

Si $\lim f = \lim g = 0$ (ou $\pm\infty$) et $g'(a) \neq 0$:
 $\lim f(x)/g(x) = \lim f'(x)/g'(x)$

[DÉMONSTRATION] $\lim_{x \rightarrow 0} \sin(x)/x = 1$

Pour $x > 0$ petit, géométriquement :
 $\sin(x) < x < \tan(x)$
 Donc : $\sin(x)/x < 1$ et $\sin(x)/x > \cos(x)$
 Quand $x \rightarrow 0$, $\cos(x) \rightarrow 1$, donc par le théorème des gendarmes :
 $\lim_{x \rightarrow 0} \sin(x)/x = 1 \quad \square$

[CODE PYTHON]

```

import math

# Approcher une limite numériquement
def limite_numerique(f, a, eps_list=None):
    if eps_list is None:
        eps_list = [0.1, 0.01, 0.001, 0.0001, 0.00001]
    print(f"Limite de f en x = {a} :")
    for eps in eps_list:
        try:
            val_droite = f(a + eps)
            val_gauche = f(a - eps)
            print(f" x={a+eps:.5f} : f(x)={val_droite:.8f} | "
                  f"x={a-eps:.5f} : f(x)={val_gauche:.8f}")
        except (ZeroDivisionError, ValueError):
            print(f" x={a+eps} : non défini")

#  $\lim_{x \rightarrow 0} \sin(x)/x$ 
limite_numerique(lambda x: math.sin(x)/x if x != 0 else None, 0)
# Converge vers 1 ✓

#  $\lim_{x \rightarrow \infty} (1 + 1/x)^x \rightarrow e$ 
print("\n(1+1/x)^x pour x grand :")
for x in [10, 100, 1000, 10000, 100000]:
    print(f" x={x:>6} : (1+1/x)^x = {(1+1/x)**x:.8f}")
print(f" e = {math.e:.8f}")

# Tester la continuité

```

```
def est_continue_en(f, a, delta=1e-6):
    """Vérifie numériquement si f est continue en a"""
    try:
        fa = f(a)
        lim_gauche = f(a - delta)
        lim_droite = f(a + delta)
        return abs(lim_gauche - fa) < 1e-4 and abs(lim_droite - fa) < 1e-4
    except:
        return False

print("\nContinuité de 1/x en x=0 :", est_continue_en(lambda x: 1/x, 0)) # False
print("Continuité de x² en x=2 :", est_continue_en(lambda x: x**2, 2)) # True
```

[PIÈGE]

$\lim_{x \rightarrow 0} f(x)$ peut exister même si $f(0)$ n'est pas défini.
 Et même si $f(0)$ est défini, la limite peut être différente de $f(0)$.
 La continuité exige LES DEUX : existence de la limite ET égalité avec $f(a)$.

7. DÉRIVATION

[THÉORIE]

La dérivée $f'(x)$ mesure la VITESSE DE VARIATION de f en x .
 C'est la pente de la tangente à la courbe en ce point.
 En physique : si $f(t)$ = position, alors $f'(t)$ = vitesse.
 C'est l'outil fondamental pour trouver les maxima/minima.

[DÉFINITION]

$f'(a) = \lim_{h \rightarrow 0} (f(a+h) - f(a)) / h$
 = taux de variation instantané en a

FORMULES DE RÉFÉRENCE :

$(x^n)' = n \cdot x^{n-1}$
 $(e^x)' = e^x$
 $(\ln x)' = 1/x$
 $(\sin x)' = \cos x$
 $(\cos x)' = -\sin x$

RÈGLES DE CALCUL :

$(f + g)' = f' + g'$
 $(f \cdot g)' = f'g + fg'$ (règle du produit)
 $(f/g)' = (f'g - fg')/g^2$ (règle du quotient)
 $(f \circ g)' = g' \cdot (f' \circ g)$ (règle de composition / chaîne)

APPLICATIONS :

$f'(a) = 0 \rightarrow$ point critique (possible max/min)
 $f' > 0 \rightarrow f$ croissante | $f' < 0 \rightarrow f$ décroissante
 $f'' > 0 \rightarrow$ convexe | $f'' < 0 \rightarrow$ concave

[DÉMONSTRATION] Dériver x^n à partir de la définition

$f(x) = x^n$.
 $f'(x) = \lim_{h \rightarrow 0} ((x+h)^n - x^n) / h$
 Binôme de Newton : $(x+h)^n = x^n + n \cdot x^{n-1} \cdot h + \text{termes en } h^2 \text{ et plus}$
 Donc $(f(x+h) - f(x)) / h = n \cdot x^{n-1} + \text{termes en } h \rightarrow 0$
 Conclusion : $f'(x) = n \cdot x^{n-1}$ □

[CODE PYTHON]

```
import math

# Dérivée numérique (différence centrée, plus précise)
def derivee(f, x, h=1e-7):
    return (f(x + h) - f(x - h)) / (2 * h)

# Tester les formules
f = lambda x: x**3
print("f(x)=x³, f'(2) =", derivee(f, 2)) # devrait être 3×4=12

g = lambda x: math.exp(x)
print("f(x)=eˣ, f'(1) =", derivee(g, 1)) # devrait être e≈2.718

h_func = lambda x: math.sin(x)
```



```
print("f(x)=sin(x), f'(π/4) =", derivee(h_func, math.pi/4)) # cos(π/4)≈0.707
```

Trouver les extremums d'un polynôme

```
def trouver_extremums(f, x_min, x_max, n=10000):
    """Trouve les points où f' ≈ 0"""
    xs = [x_min + i*(x_max-x_min)/n for i in range(n)]
    extremums = []
    for i in range(1, len(xs)-1):
        d = derivee(f, xs[i])
        if abs(d) < 1e-3:
            extremums.append((round(xs[i], 3), round(f(xs[i]), 3)))
    return extremums
```

```
# f(x) = x³ - 3x (minimum en x=1, maximum en x=-1)
poly = lambda x: x**3 - 3*x
exts = trouver_extremums(poly, -3, 3)
print("\nExtrémums de x³-3x :", exts[5:]) # ≈ (-1, 2) et (1, -2)
```

[PIÈGE]

La dérivée n° de $(f \cdot g) \neq f^{(n)} \cdot g^{(n)}$.
On utilise la formule de Leibniz pour les dérivées d'ordre supérieur
de produits. Erreur très fréquente !

8. INTÉGRATION

[THÉORIE]

L'intégrale calcule des AIRES sous une courbe. C'est l'opération inverse
de la dérivée. En physique : si $f'(t)$ = vitesse, alors $\int f' dt$ = position.
L'intégrale définie $\int [a,b] f(x) dx$ donne l'aire entre la courbe et l'axe x.

[DÉFINITION]

PRIMITIVE : F est une primitive de f si $F'(x) = f(x)$
INTÉGRALE INDÉFINIE : $\int f(x) dx = F(x) + C$ (C constante quelconque)
INTÉGRALE DÉFINIE : $\int [a,b] f(x) dx = F(b) - F(a)$ (Théorème fondamental)

PRIMITIVES DE RÉFÉRENCE :

$$\begin{aligned}\int x^n dx &= x^{n+1}/(n+1) + C & (n \neq -1) \\ \int e^x dx &= e^x + C \\ \int (1/x) dx &= \ln|x| + C \\ \int \sin x dx &= -\cos x + C \\ \int \cos x dx &= \sin x + C\end{aligned}$$

TECHNIQUES :

Intégration par parties : $\int u \cdot v' dx = u \cdot v - \int u' \cdot v dx$
Substitution : changer de variable $x \rightarrow u(x)$

[DÉMONSTRATION] $\int [0,1] x^2 dx = 1/3$

La primitive de x^2 est $F(x) = x^3/3$.

$\int [0,1] x^2 dx = F(1) - F(0) = 1/3 - 0 = 1/3 \quad \square$

Interprétation : l'aire sous la parabole $y=x^2$ entre $x=0$ et $x=1$ vaut $1/3$.

[CODE PYTHON]

```
import math
```

Intégration numérique – méthode des trapèzes

```
def integrale_trapezes(f, a, b, n=10000):
    """∫[a,b] f(x)dx par méthode des trapèzes"""
    h = (b - a) / n
    total = (f(a) + f(b)) / 2
    for i in range(1, n):
        total += f(a + i*h)
    return total * h
```

```
# Tester ∫[0,1] x² dx
print("∫[0,1] x² dx =", integrale_trapezes(lambda x: x**2, 0, 1))
# ≈ 0.3333 (exact = 1/3)
```

```
# ∫[0,π] sin(x) dx = 2
print("∫[0,π] sin(x) dx =", integrale_trapezes(math.sin, 0, math.pi))
```

```
# ≈ 2.0

# Méthode de Simpson (plus précise)
def integrale_simpson(f, a, b, n=1000):
    """n doit être pair"""
    h = (b - a) / n
    total = f(a) + f(b)
    for i in range(1, n):
        coeff = 4 if i % 2 else 2
        total += coeff * f(a + i*h)
    return total * h / 3

print("\nSimpson  $\int[0,1] e^x dx =$ ", integrale_simpson(math.exp, 0, 1))
print("Valeur exacte :  $e - 1 =$ ", math.e - 1) # ≈ 1.71828

# Approximation de  $\pi$  par intégration
#  $\int[0,1] \frac{4}{(1+x^2)} dx = \pi$ 
pi_approx = integrale_simpson(lambda x: 4/(1+x**2), 0, 1)
print(f"\n $\pi \approx \{pi\_approx:.8f\}$  (exact:  $\{math.pi:.8f\}$ )")
```

[PIÈGE]
 $\int[a,b] f(x)dx$ peut être NÉGATIF si $f(x) < 0$ sur $[a,b]$.
 L'aire "géométrique" toujours positive nécessite $\int|f(x)|dx$.
 Ne pas confondre "aire algébrique" et "aire géométrique" !

9. SUITES NUMÉRIQUES

[THÉORIE]
 Une suite, c'est une liste ordonnée de nombres : $u_0, u_1, u_2, \dots$
 On s'intéresse à son comportement quand $n \rightarrow +\infty$: converge-t-elle ?
 Diverge-t-elle ? Les suites sont partout : algorithmes itératifs,
 intérêts composés, fractales...

[DÉFINITION]
 SUITE ARITHMÉTIQUE : $u_{n+1} = u_n + r \iff u_n = u_0 + n \cdot r$
 Somme : $S = n(u_0 + u_{n-1})/2$

SUITE GÉOMÉTRIQUE : $u_{n+1} = q \cdot u_n \iff u_n = u_0 \cdot q^n$
 Somme : $S = u_0 \cdot (1 - q^n)/(1 - q)$ si $q \neq 1$

CONVERGENCE : $\lim_{n \rightarrow \infty} u_n = L$ (la suite "se stabilise" vers L)
 $|q| < 1 \iff q^n \rightarrow 0$ (géométrie converge)
 $|q| > 1 \iff q^n \rightarrow \infty$ (géométrie diverge)

[DÉMONSTRATION] Suite de Fibonacci : $F_n \approx \phi^n/\sqrt{5}$ où $\phi = (1+\sqrt{5})/2$
 Posons $F_n = A\phi^n + B\psi^n$ où $\phi = (1+\sqrt{5})/2$, $\psi = (1-\sqrt{5})/2$.
 Les deux vérifient $x^2 = x + 1$ (équation car. de $F_n = F_{n-1} + F_{n-2}$).
 Conditions initiales $F_0=0, F_1=1$ donnent $A=1/\sqrt{5}$, $B=-1/\sqrt{5}$.
 Donc $F_n = (\phi^n - \psi^n)/\sqrt{5}$. Comme $|\psi| < 1$, $\psi^n \rightarrow 0$, donc $F_n \approx \phi^n/\sqrt{5}$. $\square$

[CODE PYTHON]

```
import math

# Suite arithmétique
u0, r, n = 3, 5, 10
suite_arith = [u0 + k*r for k in range(n)]
print("Arithmétique :", suite_arith)
print("Somme :", sum(suite_arith), "=", n*(suite_arith[0]+suite_arith[-1])/2)

# Suite géométrique
u0, q, n = 1, 2, 10
suite_geom = [u0 * q**k for k in range(n)]
print("\nGéométrie :", suite_geom)

# Convergence :  $(0.5)^n \rightarrow 0$ 
print("\n $(0.5)^n$  converge :")
for k in [1, 5, 10, 20, 50]:
    print(f"  n={k:>2} :  $(0.5)^n = \{0.5**k:.10f\}$ ")

# Suite de Fibonacci
```

```
def fibonacci(n):
    a, b = 0, 1
    suite = [a, b]
    for _ in range(n-2):
        a, b = b, a+b
        suite.append(b)
    return suite

fibs = fibonacci(15)
print("\nFibonacci :", fibs)

# Vérifier  $F_n \approx \phi^n / \sqrt{5}$ 
phi = (1 + math.sqrt(5)) / 2
print("\nFib(15) exact :", fibs[14])
print("Formule  $\phi^n / \sqrt{5}$  :", round(phi**14 / math.sqrt(5)))

# Ratio successifs  $\rightarrow \phi$  (nombre d'or)
ratios = [fibs[i+1]/fibs[i] for i in range(5, 14)]
print("\nRatios  $F_{n+1}/F_n \rightarrow$ ", [f"{r:.5f}" for r in ratios])
print(" $\phi =$ ", f"{phi:.5f}")
```

[PIÈGE]

Une suite bornée et monotone CONVERGE (théorème de la limite monotone).
 Mais une suite bornée seule ne converge pas forcément !
 Ex : $u_n = (-1)^n$ est bornée mais oscille entre +1 et -1 $\rightarrow$ diverge.

BLOC C – TRIGONOMÉTRIE & GÉOMÉTRIE

10. TRIGONOMÉTRIE – le cercle trigonométrique

[THÉORIE]

La trigonométrie, c'est l'étude des relations dans les triangles et des fonctions périodiques liées aux angles.
 Le cercle trigonométrique (rayon 1, centré en 0) est le support visuel de tout : $\cos$ = abscisse, $\sin$ = ordonnée du point sur le cercle.

[DÉFINITION]

Pour un angle θ (en radians) :
 $\cos(\theta)$ = abscisse du point sur le cercle unité
 $\sin(\theta)$ = ordonnée du point sur le cercle unité
 $\tan(\theta) = \sin(\theta)/\cos(\theta)$ (défini si $\cos(\theta) \neq 0$)

RELATION FONDAMENTALE : $\cos^2(\theta) + \sin^2(\theta) = 1$

VALEURS REMARQUABLES :

$\theta = 0$	: $\cos=1$,	$\sin=0$,	$\tan=0$
$\theta = \pi/6$	: $\cos=\sqrt{3}/2$,	$\sin=1/2$,	$\tan=1/\sqrt{3}$
$\theta = \pi/4$	: $\cos=\sqrt{2}/2$,	$\sin=\sqrt{2}/2$,	$\tan=1$
$\theta = \pi/3$	: $\cos=1/2$,	$\sin=\sqrt{3}/2$,	$\tan=\sqrt{3}$
$\theta = \pi/2$	: $\cos=0$,	$\sin=1$,	$\tan=\text{indéfini}$

PÉRIODICITÉ :

$\sin$ et $\cos$ sont 2π -périodiques
 $\tan$ est π -périodique

[DÉMONSTRATION] $\cos^2(\theta) + \sin^2(\theta) = 1$

Par définition, $(\cos \theta, \sin \theta)$ est un point sur le cercle de rayon 1.
 La distance au centre vaut 1.
 $\text{Distance}^2 = \cos^2\theta + \sin^2\theta = 1^2 = 1 \quad \square$

[CODE PYTHON]

```
import math

# Valeurs de référence
angles = [0, math.pi/6, math.pi/4, math.pi/3, math.pi/2, math.pi]
noms = ["0", "π/6", "π/4", "π/3", "π/2", "π"]

print(f"{'Angle':<8} {'cos':<10} {'sin':<10} {'tan':<10}")
```

```

print("-" * 40)
for a, nom in zip(angles, noms):
    cos_a = round(math.cos(a), 4)
    sin_a = round(math.sin(a), 4)
    tan_a = round(math.tan(a), 4) if abs(math.cos(a)) > 1e-10 else "-"
    print(f"{nom:<8} {cos_a:<10} {sin_a:<10} {tan_a}")

# Vérifier cos²+sin²=1
print("\nVérification cos²+sin²=1 :")
for a in [0.3, 1.2, 2.5, 3.7]:
    val = math.cos(a)**2 + math.sin(a)**2
    print(f"  θ={a:.1f}: cos²+sin² = {val:.10f}") # Toujours 1

# Conversion degrés ↔ radians
def deg_vers_rad(deg):
    return deg * math.pi / 180

def rad_vers_deg(rad):
    return rad * 180 / math.pi

print(f"\n45° = {deg_vers_rad(45):.4f} rad")
print(f"π/3 rad = {rad_vers_deg(math.pi/3):.1f}°")

# Périodicité : sin(x + 2π) = sin(x)
x = 1.5
print(f"\nsin({x:.1f}) = {math.sin(x):.6f}")
print(f"sin({x:.1f}+2π) = {math.sin(x + 2*math.pi):.6f}") # Identique ✓

```

[PIÈGE]
 Python (et toutes les fonctions math) travaille en RADIANS, pas en degrés !
 math.sin(90) ≠ 1. Il faut math.sin(math.pi/2) = 1.

11. TRIGONOMETRIE AVANCÉE – formules et équations

[THÉORIE]

Les formules de trigonométrie permettent de transformer, simplifier et résoudre des équations trigonométriques. Elles sont omniprésentes en physique (ondes, oscillations), en signal et en géométrie.

[DÉFINITION]

FORMULES D'ADDITION :

```

cos(a + b) = cos a · cos b - sin a · sin b
cos(a - b) = cos a · cos b + sin a · sin b
sin(a + b) = sin a · cos b + cos a · sin b
sin(a - b) = sin a · cos b - cos a · sin b

```

FORMULES DE DUPLICATION :

```

cos(2a) = cos²a - sin²a = 2cos²a - 1 = 1 - 2sin²a
sin(2a) = 2 sin a · cos a

```

FORMULES DE LINEARISATION :

```

cos²a = (1 + cos 2a)/2
sin²a = (1 - cos 2a)/2

```

ÉQUATIONS TRIGONOMETRIQUES :

```

sin(x) = k  ⇔  x = arcsin(k) + 2kπ  ou  x = π - arcsin(k) + 2kπ
cos(x) = k  ⇔  x = ±arccos(k) + 2kπ

```

[DÉMONSTRATION] cos(2a) = 1 - 2sin²(a)

```

cos(2a) = cos(a+a) = cos²a - sin²a
Or cos²a = 1 - sin²a (relation fondamentale)
Donc cos(2a) = (1-sin²a) - sin²a = 1 - 2sin²a  □

```

[CODE PYTHON]

```

import math

# Vérification des formules d'addition
a, b = 0.6, 1.1

print("Formules d'addition :")

```

```

print(f"cos(a+b) = {math.cos(a+b):.6f}")
print(f"cos a·cos b - sin a·sin b = "
      f"{math.cos(a)*math.cos(b) - math.sin(a)*math.sin(b):.6f}") # Égaux ✓

print(f"\nsin(a+b) = {math.sin(a+b):.6f}")
print(f"sin a·cos b + cos a·sin b = "
      f"{math.sin(a)*math.cos(b) + math.cos(a)*math.sin(b):.6f}") # Égaux ✓

# Formule de duplication
print("\nFormules de duplication :")
a = 0.8
print(f"sin(2a) = {math.sin(2*a):.6f}")
print(f"2·sin(a)·cos(a) = {2*math.sin(a)*math.cos(a):.6f}") # Égaux ✓

# Résoudre sin(x) = 0.5 dans [0, 2π]
val = 0.5
x1 = math.asin(val) # solution principale
x2 = math.pi - x1 # deuxième solution
print(f"\nSolutions de sin(x) = {val} dans [0, 2π]:")
print(f"  x1 = {x1:.4f} rad = {math.degrees(x1):.1f}°")
print(f"  x2 = {x2:.4f} rad = {math.degrees(x2):.1f}°")
print(f"  Vérification : sin(x1)={math.sin(x1):.4f}, sin(x2)={math.sin(x2):.4f}")

```

[PIÈGE]

arcsin est défini sur $[-\pi/2, \pi/2]$: il ne donne QUE la solution principale.
 Pour $\sin(x) = 0.5$, il y a UNE INFINITÉ de solutions $(+ 2k\pi)$!
 Toujours vérifier l'ensemble des solutions, pas juste arcsin.

12. GÉOMÉTRIE ANALYTIQUE – droites et cercles

[THÉORIE]

La géométrie analytique traduit les objets géométriques (droites, cercles) en ÉQUATIONS algébriques. Descartes (1637) a eu cette idée géniale :
 un point = un couple de coordonnées, une figure = une équation.

[DÉFINITION]

DROITE :

Forme pente-ordonnée : $y = mx + b$ (m = pente, b = ordonnée à l'origine)
 Forme générale : $ax + by + c = 0$
 Pente entre (x_1, y_1) et (x_2, y_2) : $m = (y_2 - y_1) / (x_2 - x_1)$

DISTANCE point-droite :

$d(P, D) = |ax_0 + by_0 + c| / \sqrt{a^2 + b^2}$ pour $P=(x_0, y_0)$ et $D: ax+by+c=0$

CERCLE :

Centre (h, k) , rayon r : $(x-h)^2 + (y-k)^2 = r^2$
 Forme générale : $x^2 + y^2 + Dx + Ey + F = 0$

DROITES PERPENDICULAIRES : pentes m_1 et m_2 telles que $m_1 \cdot m_2 = -1$

DROITES PARALLÈLES : pentes égales $m_1 = m_2$

[DÉMONSTRATION] Distance d'un point à une droite

Droite D : $ax + by + c = 0$. Point $P = (x_0, y_0)$.

La distance est la longueur du segment perpendiculaire de P à D .

On projette P sur D , et par calcul vectoriel on trouve :

$$d = |ax_0 + by_0 + c| / \sqrt{a^2 + b^2} \quad \square$$

[CODE PYTHON]

```

import math

def pente(x1, y1, x2, y2):
    if x2 == x1:
        return float('inf')
    return (y2 - y1) / (x2 - x1)

def droite_deux_points(x1, y1, x2, y2):
    """Retourne (m, b) tels que y = mx + b"""
    m = pente(x1, y1, x2, y2)
    b = y1 - m * x1
    return m, b

```

```

def distance_point_droite(x0, y0, a, b, c):
    """Distance de (x0,y0) à ax + by + c = 0"""
    return abs(a*x0 + b*y0 + c) / math.sqrt(a**2 + b**2)

# Exemple
m, b = droite_deux_points(1, 2, 4, 8)
print(f"Droite par (1,2) et (4,8) : y = {m:.2f}x + {b:.2f}")
# y = 2x + 0

# Distance du point (0, 3) à la droite y = 2x soit -2x + y = 0
d = distance_point_droite(0, 3, -2, 1, 0)
print(f"Distance de (0,3) à y=2x : {d:.4f}")

# Perpendiculaire à y=2x passant par (0,3)
m_perp = -1/2 # m1 * m2 = -1
b_perp = 3 - m_perp * 0
print(f"Perpendiculaire : y = {m_perp}x + {b_perp}")

# Intersection de deux droites
def intersection(m1, b1, m2, b2):
    """Intersection de y=m1x+b1 et y=m2x+b2"""
    if m1 == m2:
        return None # Parallèles
    x = (b2 - b1) / (m1 - m2)
    y = m1 * x + b1
    return round(x, 4), round(y, 4)

print(f"Intersection : {intersection(2, 0, -0.5, 3)}")

```

[PIÈGE]

La pente m d'une droite verticale est infinie (non définie).
Il faut traiter ce cas séparément ! x = constante n'a pas de pente.

13. GÉOMÉTRIE VECTORIELLE

[THÉORIE]

Un vecteur, c'est une FLÈCHE : il a une direction, un sens et une longueur.
On l'utilise pour représenter des déplacements, des forces, des vitesses.
Le produit scalaire permet de calculer des angles et des projections.

[DÉFINITION]

VECTEUR : $\vec{u} = (u_x, u_y)$ en 2D, ou (u_x, u_y, u_z) en 3D

NORME : $||\vec{u}|| = \sqrt{u_x^2 + u_y^2}$

ADDITION : $\vec{u} + \vec{v} = (u_x+v_x, u_y+v_y)$

PRODUIT SCALAIRE : $\vec{u} \cdot \vec{v} = u_x v_x + u_y v_y = ||\vec{u}|| \cdot ||\vec{v}|| \cdot \cos(\theta)$
Si $\vec{u} \cdot \vec{v} = 0$ $\vec{u} \perp \vec{v}$ (vecteurs orthogonaux)

PRODUIT VECTORIEL (3D) :

$\vec{u} \times \vec{v} = (u_y v_z - u_z v_y, u_z v_x - u_x v_z, u_x v_y - u_y v_x)$
 $||\vec{u} \times \vec{v}|| = ||\vec{u}|| \cdot ||\vec{v}|| \cdot \sin(\theta) = \text{aire du parallélogramme}$

[DÉMONSTRATION] L'angle entre deux vecteurs via le produit scalaire

Par la loi des cosinus dans le triangle formé par $\vec{u}$, $\vec{v}$ et $\vec{u}-\vec{v}$:

$||\vec{u} - \vec{v}||^2 = ||\vec{u}||^2 + ||\vec{v}||^2 - 2||\vec{u}|| \cdot ||\vec{v}|| \cdot \cos \theta$
Or $||\vec{u} - \vec{v}||^2 = (u_x - v_x)^2 + (u_y - v_y)^2$
 $= ||\vec{u}||^2 - 2(u_x v_x + u_y v_y) + ||\vec{v}||^2$

On identifie : $\vec{u} \cdot \vec{v} = u_x v_x + u_y v_y = ||\vec{u}|| \cdot ||\vec{v}|| \cdot \cos \theta$ □

[CODE PYTHON]

```

import math

def norme(v):
    return math.sqrt(sum(x**2 for x in v))

def produit_scalaire(u, v):
    return sum(ui*vi for ui, vi in zip(u, v))

```

```

def angle_entre(u, v):
    cos_theta = produit_scalaire(u, v) / (norme(u) * norme(v))
    cos_theta = max(-1, min(1, cos_theta)) # Clamp pour éviter erreurs numériques
    return math.degrees(math.acos(cos_theta))

def produit_vectoriel_3d(u, v):
    """Produit vectoriel en 3D"""
    return (u[1]*v[2] - u[2]*v[1],
            u[2]*v[0] - u[0]*v[2],
            u[0]*v[1] - u[1]*v[0])

# Exemples 2D
u = (3, 4)
v = (1, 2)
print(f"u = {u}, ||u|| = {norme(u)}") # 5
print(f"v = {v}, ||v|| = {norme(v):.4f}")
print(f"u · v = {produit_scalaire(u, v)}") # 11
print(f"Angle u,v = {angle_entre(u, v):.2f}°")

# Vecteurs orthogonaux
w = (-4, 3) # Perpendiculaire à u=(3,4)
print(f"\nu · w = {produit_scalaire(u, w)}") # 0 → perpendiculaires ✓

# Produit vectoriel 3D
a = (1, 0, 0)
b = (0, 1, 0)
c = produit_vectoriel_3d(a, b)
print(f"\n(1,0,0) × (0,1,0) = {c}") # (0,0,1) ✓

# Projection de u sur v
def projection(u, v):
    """Projection de u sur v"""
    scale = produit_scalaire(u, v) / produit_scalaire(v, v)
    return tuple(scale * vi for vi in v)

print(f"\nProjection de {u} sur {v} = {projection(u, v)}")

```

[PIÈGE]

Le produit scalaire est un SCALAIRE (un nombre), pas un vecteur.
 Le produit vectoriel est un VECTEUR, mais n'existe qu'en 3D (et 7D).
 Ne pas les confondre !

BLOC D – ALGÈBRE LINÉAIRE

14. VECTEURS & ESPACES VECTORIELS

[THÉORIE]

Un espace vectoriel, c'est un ensemble de vecteurs où on peut additionner et multiplier par des scalaires, et ça reste dans l'espace.
 $\mathbb{R}^2$, $\mathbb{R}^3$, les polynômes, les fonctions continues : ce sont tous des espaces vectoriels. C'est le cadre de toute l'algèbre linéaire.

[DÉFINITION]

BASE : ensemble de vecteurs $\{v_1, \dots, v_n\}$:

- LIBRES (linéairement indépendants) : aucun n'est combinaison des autres
- GÉNÉRATEURS : tout vecteur de l'espace s'écrit comme combinaison de ces v_i

DIMENSION : nombre de vecteurs dans une base

INDÉPENDANCE LINÉAIRE : $v_1, \dots, v_n$ sont indépendants si :
 $\alpha_1 v_1 + \dots + \alpha_n v_n = 0 \iff \text{tous } \alpha_i = 0$

RANG d'une famille : nombre de vecteurs indépendants maximum

[DÉMONSTRATION] $(1,0)$ et $(0,1)$ sont une base de $\mathbb{R}^2$

Indépendance : $\alpha \cdot (1,0) + \beta \cdot (0,1) = (0,0) \iff (\alpha, \beta) = (0,0) \checkmark$

Génération : tout $(x,y) = x \cdot (1,0) + y \cdot (0,1) \checkmark$

Donc $\{(1,0), (0,1)\}$ est une base de $\mathbb{R}^2$. Dimension = 2. $\square$

[CODE PYTHON]

```
import numpy as np

# Vérifier l'indépendance linéaire via le rang
def independants(vecteurs):
    """Vrai si les vecteurs sont linéairement indépendants"""
    A = np.array(vecteurs, dtype=float)
    return np.linalg.matrix_rank(A) == len(vecteurs)

v1 = [1, 0, 0]
v2 = [0, 1, 0]
v3 = [0, 0, 1]
v4 = [1, 1, 0] # = v1 + v2 → dépendant

print("v1,v2,v3 indépendants :", independants([v1, v2, v3])) # True
print("v1,v2,v4 indépendants :", independants([v1, v2, v4])) # False !

# Exprimer un vecteur dans une base
def coords_dans_base(v, base):
    """Coordonnées de v dans la base donnée"""
    B = np.array(base, dtype=float).T
    return np.linalg.solve(B, v)

base = [[1, 1], [1, -1]] # Base non canonique de  $\mathbb{R}^2$ 
v = [3, 1]
coords = coords_dans_base(v, base)
print(f"\nCoordonnées de {v} dans la base {base} :", coords)
# Vérification
reconstitue = sum(c * np.array(b) for c, b in zip(coords, base))
print("Vérification :", reconstitue)
```

[PIÈGE]

Des vecteurs peuvent sembler "différents" mais être dépendants.
(2,4) et (1,2) sont dépendants car $(2,4) = 2 \cdot (1,2)$.
Toujours vérifier avec le rang, pas à l'œil.

15. MATRICES

[THÉORIE]

Une matrice est un tableau de nombres. Elle représente une TRANSFORMATION LINÉAIRE : rotation, homothétie, projection... Multiplier une matrice par un vecteur, c'est appliquer la transformation à ce vecteur.

[DÉFINITION]

Matrice A de taille $m \times n$: m lignes, n colonnes.
 A_{ij} = élément ligne i, colonne j.

OPÉRATIONS :

Addition $A+B$: terme à terme (même taille)
Produit $A \cdot B$: $(AB)_{ij} = \sum_k A_{ik} \cdot B_{kj}$ (A est $m \times p$, B est $p \times n$)
Transposée A^T : $(A^T)_{ij} = A_{ji}$

MATRICES PARTICULIÈRES :

Identité I_n : 1 sur la diagonale, 0 ailleurs
Symétrique : $A = A^T$
Orthogonale : $A \cdot A^T = I$ (les colonnes forment une base orthonormale)

VALEURS PROPRES λ et VECTEURS PROPRES v :

$A \cdot v = \lambda \cdot v \rightarrow$ les vecteurs que A "étire" sans changer de direction

[DÉMONSTRATION] La transposée d'un produit : $(AB)^T = B^T A^T$

$((AB)^T)_{ij} = (AB)_{ji} = \sum_k A_{jk} \cdot B_{ki}$
 $(B^T A^T)_{ij} = \sum_k (B^T)_{ik} \cdot (A^T)_{kj} = \sum_k B_{ki} \cdot A_{jk} = \sum_k A_{jk} \cdot B_{ki}$
Les deux sont égaux. $\square$

[CODE PYTHON]

```
import numpy as np
```



```

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("A =\n", A)
print("B =\n", B)
print("A + B =\n", A + B)
print("A · B =\n", A @ B)      # @ est le produit matriciel
print("AT =\n", A.T)

# Vérifier (AB)T = BTAT
print("\n(AB)T =\n", (A @ B).T)
print("BTAT =\n", B.T @ A.T) # Égaux ✓

# Valeurs et vecteurs propres
print("\nValeurs propres de A :")
valeurs, vecteurs = np.linalg.eig(A)
print("λ =", valeurs)
print("Vecteurs propres =\n", vecteurs)

# Vérification A·v = λ·v
for i in range(len(valeurs)):
    v = vecteurs[:, i]
    lam = valeurs[i]
    diff = np.linalg.norm(A @ v - lam * v)
    print(f" ||A·v{i} - λ{i}·v{i}|| = {diff:.2e}") # ≈ 0 ✓

# Matrice de rotation (rotation de θ)
def rotation(theta):
    import math
    return np.array([[math.cos(theta), -math.sin(theta)],
                     [math.sin(theta),  math.cos(theta)]])

R = rotation(math.pi/4) # Rotation de 45°
import math
v = np.array([1, 0])
print("\nRotation de (1,0) de 45° :", R @ v) # (√2/2, √2/2)

```

[PIÈGE]

Le produit matriciel n'est PAS commutatif : $A \cdot B \neq B \cdot A$ en général.
 Contrairement à la multiplication des nombres réels !

16. DÉTERMINANTS & SYSTÈMES

[THÉORIE]

Le déterminant d'une matrice est un nombre qui mesure "comment la transformation change les aires/volumes". Si $\det(A) = 0$, la matrice "écrase" l'espace dans une dimension inférieure et le système $Ax = b$ n'a pas de solution unique.

[DÉFINITION]

det 2×2 : $\begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$

det 3×3 (règle de Sarrus) :

$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = aei + bfg + cdh - ceg - bdi - afh$

PROPRIÉTÉS :

$\det(AB) = \det(A) \cdot \det(B)$
 $\det(A^T) = \det(A)$
 A inversible $\iff \det(A) \neq 0$
 A^{-1} existe $\iff \det(A) \neq 0$

RÈGLE DE CRAMER (systèmes 2×2, 3×3) :

Pour $Ax = b$, la solution est $x_i = \det(A_i) / \det(A)$
 où A_i est A avec la colonne i remplacée par b .

[DÉMONSTRATION] $\det(A)=0 \iff$ colonnes linéairement dépendantes (cas 2×2)

$A = \begin{bmatrix} u & v \end{bmatrix}$ colonnes. $\det(A) = u_1v_2 - u_2v_1$.
 Si $v = \lambda u$: $\det = u_1(\lambda u_2) - u_2(\lambda u_1) = \lambda(u_1u_2 - u_2u_1) = 0 \quad \checkmark$

Réciproquement, si $\det=0$ et $u \neq 0$: $v_1/u_1 = v_2/u_2 = \lambda \rightarrow v = \lambda u \checkmark \square$

[CODE PYTHON]

```
import numpy as np

A = np.array([[2, 1], [5, 3]])
print("det(A) =", np.linalg.det(A)) # 2*3 - 1*5 = 1

B = np.array([[1, 2], [2, 4]])
print("det(B) =", np.linalg.det(B)) # 1*4 - 2*2 = 0 (colonnes dépendantes !)
```

Règle de Cramer 2x2

```
def cramer_2x2(A, b):
    det_A = np.linalg.det(A)
    if abs(det_A) < 1e-10:
        return None # Pas de solution unique
    A1 = A.copy(); A1[:, 0] = b
    A2 = A.copy(); A2[:, 1] = b
    x1 = np.linalg.det(A1) / det_A
    x2 = np.linalg.det(A2) / det_A
    return x1, x2

A = np.array([[2., 1.], [5., 3.]])
b = np.array([8., 19.])
sol = cramer_2x2(A, b)
print(f"\nSolution par Cramer : x={sol[0]:.2f}, y={sol[1]:.2f}")
print(f"Vérification : A@x = {A @ np.array(sol)}, b = {b}")
```

Inverse via det (théorème de Cramer généralisé)

```
A_inv = np.linalg.inv(A)
print(f"\nA-1 =\n{A_inv}")
print(f"A·A-1 =\n{np.round(A @ A_inv, 4)}") # Identité ✓
```

[PIÈGE]

$\det(A + B) \neq \det(A) + \det(B)$. Le déterminant n'est PAS linéaire sur la somme de matrices. C'est $\det(AB) = \det(A) \cdot \det(B)$ (produit, pas somme).

BLOC E – PROBABILITÉS & STATISTIQUES

17. PROBABILITÉS – fondamentaux

[THÉORIE]

La probabilité mesure le degré de certitude qu'un événement se produise. 0 = impossible, 1 = certain. Elle formalise l'incertitude en une valeur numérique manipulable. Base de la statistique, de la finance, de l'IA.

[DÉFINITION]

ESPACE PROBABILISÉ : (Ω, F, P) où
 Ω = espace des possibles (ensemble de tous les résultats)
 F = événements (sous-ensembles de Ω)
 $P : F \rightarrow [0,1]$ telle que $P(\Omega) = 1$

AXIOMES DE KOLMOGOROV :

1. $P(A) \geq 0$
2. $P(\Omega) = 1$
3. $A \cap B = \emptyset \Rightarrow P(A \cup B) = P(A) + P(B)$ (additivité)

FORMULES FONDAMENTALES :

$P(\bar{A}) = 1 - P(A)$ (événement complémentaire)
 $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ (inclusion-exclusion)
 $P(A \cap B) = P(A|B) \cdot P(B)$ (probabilité conditionnelle)

[DÉMONSTRATION] $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

Partitionner $A \cup B$ en ensembles disjoints :

$A \cup B = (A \setminus B) \cup (A \cap B) \cup (B \setminus A)$ (partition de $A \cup B$)

Par additivité :

$P(A \cup B) = P(A \setminus B) + P(A \cap B) + P(B \setminus A)$

Or $P(A) = P(A \setminus B) + P(A \cap B)$ donc $P(A \setminus B) = P(A) - P(A \cap B)$. Idem pour B.

$$\begin{aligned}\text{Donc } P(A \cup B) &= P(A) - P(A \cap B) + P(A \cap B) + P(B) - P(A \cap B) \\ &= P(A) + P(B) - P(A \cap B) \quad \square\end{aligned}$$

[CODE PYTHON]

```
import random
from collections import Counter

# Simulation Monte-Carlo : probabilité empirique
def simulation(experience, n_essais=100000):
    """Simule n_essais et retourne les fréquences"""
    resultats = [experience() for _ in range(n_essais)]
    return Counter(resultats)

# Lancer un dé
def lancer_de():
    return random.randint(1, 6)

freq = simulation(lancer_de)
print("Fréquences observées sur 100 000 lancers :")
for face in sorted(freq):
    p_obs = freq[face] / 100000
    print(f"Face {face} : {p_obs:.4f} (théorique : {1/6:.4f})")

# Probabilité d'obtenir un 6 OU un nombre pair
n = 100000
compte = sum(1 for _ in range(n)
              if lancer_de() in {6, 2, 4} or lancer_de() == 6)
# P(pair u {6}) = P(pair) + P({6}) - P(pair n {6}) = 1/2 + 1/6 - 1/6 = 1/2

# Formule P(AuB)
PA = 3/6 # Pair : {2,4,6}
PB = 1/6 # {6}
PAinterB = 1/6 # pair ET 6 → juste {6}
print(f"\nP(pair u {{6}}) = {PA + PB - PAinterB:.4f}") # 3/6 = 0.5
```

[PIÈGE]

$P(A \text{ et } B) \neq P(A) \times P(B)$ sauf si A et B sont INDÉPENDANTS !
L'indépendance doit être vérifiée, pas supposée.

18. PROBABILITÉS CONDITIONNELLES & THÉORÈME DE BAYES

[THÉORIE]

"Sachant que B s'est produit, quelle est la probabilité de A ?"
C'est $P(A|B)$: la probabilité CONDITIONNELLE.
Le théorème de Bayes permet d'inverser : connaissant $P(B|A)$,
calculer $P(A|B)$. C'est la base du machine learning bayésien.

[DÉFINITION]

$P(A|B) = P(A \cap B) / P(B)$ (défini si $P(B) > 0$)

INDÉPENDANCE : A et B sont indépendants $\square P(A|B) = P(A)$
 $\square P(A \cap B) = P(A) \cdot P(B)$

THÉORÈME DES PROBABILITÉS TOTALES :

Si $B_1, \dots, B_n$ partitionnent Ω (B_i disjoints, de réunion Ω) :

$$P(A) = \sum_i P(A|B_i) \cdot P(B_i)$$

THÉORÈME DE BAYES :

$$\begin{aligned}P(B_i|A) &= P(A|B_i) \cdot P(B_i) / P(A) \\ &= P(A|B_i) \cdot P(B_i) / \sum_j P(A|B_j) \cdot P(B_j)\end{aligned}$$

[DÉMONSTRATION] Théorème de Bayes

$P(B|A) = P(A \cap B) / P(A) = P(A|B) \cdot P(B) / P(A)$
En développant $P(A)$ avec les probabilités totales :
 $P(B|A) = P(A|B) \cdot P(B) / [P(A|B) \cdot P(B) + P(A|B^c) \cdot P(B^c)] \quad \square$

[EXEMPLE] Test médical

Maladie M : prévalence $P(M) = 0.01$ (1% de la population)
Test T+ : $P(T+|M) = 0.95$ (sensibilité)
 $P(T+|M^c) = 0.05$ (faux positifs)

Si le test est positif, quelle est la probabilité d'être malade ?

```
P(M|T+) = P(T+|M) · P(M) / P(T+)
P(T+) = 0.95×0.01 + 0.05×0.99 = 0.0095 + 0.0495 = 0.059
P(M|T+) = 0.0095/0.059 ≈ 0.161 (seulement 16% !)
```

[CODE PYTHON]

```
def bayes(p_a_sachant_b, p_b, p_a_sachant_b_complementaire):
    """
    Calcule P(B|A) par Bayes.
    p_a_sachant_b = P(A|B)
    p_b = P(B) (prior)
    """
    p_b_comp = 1 - p_b
    p_a = p_a_sachant_b * p_b + p_a_sachant_b_complementaire * p_b_comp
    p_b_sachant_a = (p_a_sachant_b * p_b) / p_a
    return p_b_sachant_a, p_a

# Test médical
sensibilite = 0.95      # P(T+|M)
faux_positif = 0.05     # P(T+|M̄)
prevalence = 0.01      # P(M)

proba_malade, proba_test_pos = bayes(sensibilite, prevalence, faux_positif)
print(f"P(M|T+) = {proba_malade:.4f} = {proba_malade*100:.1f}%")
print(f"P(T+) = {proba_test_pos:.4f}")
# Résultat : seulement ~16% d'être malade malgré test positif !

# Simulation pour vérifier
import random
def simuler_test_medical(n=1000000):
    malades_detectes = 0
    tests_positifs = 0
    for _ in range(n):
        malade = random.random() < 0.01
        if malade:
            test_pos = random.random() < 0.95
        else:
            test_pos = random.random() < 0.05
        if test_pos:
            tests_positifs += 1
        if malade:
            malades_detectes += 1
    return malades_detectes / tests_positifs if tests_positifs > 0 else 0

p_sim = simuler_test_medical()
print(f"\nSimulation : P(M|T+) ≈ {p_sim:.4f}") # ≈ 0.161
```

[PIÈGE]

LE SOPHISME DE LA CONFUSION DES PROBABILITÉS INVERSES :

$P(T+|M) = 0.95$ ne veut PAS dire $P(M|T+) = 0.95$.

La prévalence $P(M)$ joue un rôle crucial. C'est la "base rate fallacy".

19. VARIABLES ALÉATOIRES DISCRÈTES

[THÉORIE]

Une variable aléatoire X associe un nombre à chaque issue possible.

"Discrete" signifie que X prend des valeurs dans un ensemble dénombrable

$\{0, 1, 2, \dots\}$. On décrit X par sa loi : quelle probabilité pour chaque valeur ?

[DÉFINITION]

LOI (distribution) : $P(X = k)$ pour chaque valeur k possible.

ESPÉRANCE : $E[X] = \sum_k k \cdot P(X=k)$ (valeur "moyenne" théorique)

VARIANCE : $\text{Var}(X) = E[X^2] - E[X]^2$ (dispersion)

ÉCART-TYPE : $\sigma = \sqrt{\text{Var}(X)}$

LOIS CLASSIQUES :

BERNOULLI $B(p)$: 1 essai, succès avec prob p

$P(X=1)=p$, $P(X=0)=1-p$, $E[X]=p$, $\text{Var}(X)=p(1-p)$

BINOMIALE $B(n,p)$: n essais indépendants Bernoulli

$$P(X=k) = C(n,k) \cdot p^k \cdot (1-p)^{n-k}$$

$$E[X] = np, \quad \text{Var}(X) = np(1-p)$$

POISSON $P(\lambda)$: nombre d'événements rares en un intervalle

$$P(X=k) = e^{-\lambda} \cdot \lambda^k / k!$$

$$E[X] = \text{Var}(X) = \lambda$$

[DÉMONSTRATION] $E[X] = np$ pour la loi binomiale

$$E[X] = \sum_{k=0}^n k \cdot C(n,k) \cdot p^k \cdot (1-p)^{n-k}$$

On utilise $k \cdot C(n,k) = n \cdot C(n-1, k-1)$.

$$\begin{aligned} E[X] &= n \cdot p \cdot \sum_{k=1}^n C(n-1, k-1) \cdot p^{k-1} \cdot (1-p)^{n-k} \\ &= n \cdot p \cdot (p + (1-p))^{n-1} \quad (\text{binôme avec } k'=k-1, n'=n-1) \\ &= n \cdot p \cdot 1 = np \quad \square \end{aligned}$$

[CODE PYTHON]

```
import math
import random

# Loi binomiale
def binomiale_proba(n, p, k):
    """P(X=k) pour X ~ B(n,p)"""
    return math.comb(n, k) * p**k * (1-p)**(n-k)

def binomiale_loi(n, p):
    return {k: binomiale_proba(n, p, k) for k in range(n+1)}

# Exemple : 10 lancers d'une pièce équilibrée
loi = binomiale_loi(10, 0.5)
print("Loi binomiale B(10, 0.5) :")
for k, prob in loi.items():
    barre = "█" * int(prob * 100)
    print(f" P(X={k}>2}) = {prob:.4f} {barre}")

# Espérance et variance
E = sum(k * p for k, p in loi.items())
E2 = sum(k**2 * p for k, p in loi.items())
Var = E2 - E**2
print(f"\nE[X] = {E:.2f} (théorique : np = {10*0.5})")
print(f"Var = {Var:.2f} (théorique : np(1-p) = {10*0.5*0.5})")

# Loi de Poisson
def poisson_proba(lam, k):
    """P(X=k) pour X ~ P(λ)"""
    return math.exp(-lam) * lam**k / math.factorial(k)

lam = 3 # Ex: 3 appels par minute en moyenne
print(f"\nLoi Poisson P(λ={lam}) - P(X=k) :")
for k in range(10):
    print(f" P(X={k}) = {poisson_proba(lam, k):.4f}")

print(f"E[X] = Var(X) = λ = {lam}")
```

[PIÈGE]

La loi de Poisson s'utilise pour des événements RARES et INDÉPENDANTS.
Si les événements ne sont pas indépendants (contagion, effet de groupe),
la Poisson ne s'applique pas !

20. VARIABLES ALÉATOIRES CONTINUES

[THÉORIE]

Une variable aléatoire continue peut prendre n'importe quelle valeur réelle dans un intervalle. On la décrit par une DENSITÉ de probabilité f (comme un histogramme infiniment fin).

La loi normale est la plus importante : elle modélise les erreurs, les mesures biologiques, les QI, etc.

[DÉFINITION]

DENSITÉ $f(x)$: $P(a \leq X \leq b) = \int [a,b] f(x) dx$, avec $\int f(x)dx = 1$
ESPÉRANCE : $E[X] = \int x \cdot f(x) dx$

VARIANCE : $\text{Var}(X) = \int (x - E[X])^2 \cdot f(x) dx$

LOI UNIFORME $U([a,b])$: $f(x) = 1/(b-a)$ sur $[a,b]$, 0 ailleurs
 $E[X] = (a+b)/2$, $\text{Var}(X) = (b-a)^2/12$

LOI NORMALE $N(\mu, \sigma^2)$:
 $f(x) = (1/\sigma\sqrt{2\pi}) \cdot \exp(-(x-\mu)^2/(2\sigma^2))$
 $E[X] = \mu$, $\text{Var}(X) = \sigma^2$
Règle empirique :
 $P(\mu-\sigma \leq X \leq \mu+\sigma) \approx 68\%$
 $P(\mu-2\sigma \leq X \leq \mu+2\sigma) \approx 95\%$
 $P(\mu-3\sigma \leq X \leq \mu+3\sigma) \approx 99.7\%$

LOI EXPONENTIELLE $E(\lambda)$: modélise des durées d'attente
 $f(x) = \lambda \cdot e^{-\lambda x}$ pour $x \geq 0$
 $E[X] = 1/\lambda$, $\text{Var}(X) = 1/\lambda^2$

[DÉMONSTRATION] $\int_{-\infty,+\infty} e^{-(x^2/2)} dx = \sqrt{2\pi}$ (l'intégrale de Gauss)
Posons $I = \int_{-\infty,+\infty} e^{-(x^2/2)} dx$.
 $I^2 = \iint e^{-(x^2+y^2)/2} dx dy$
En coordonnées polaires $x = r \cdot \cos \theta$, $y = r \cdot \sin \theta$:
 $I^2 = \int_{[0,2\pi]} \int_{[0,+\infty]} e^{-(r^2/2)} r dr d\theta = 2\pi \cdot [-e^{-(r^2/2)}]_0^\infty = 2\pi$
Donc $I = \sqrt{2\pi}$. Ceci normalise la densité normale. $\square$

[CODE PYTHON]

```
import math
import random

# Simulation loi normale par la méthode de Box-Muller
def normale(mu=0, sigma=1):
    """Génère une valeur de N(mu, sigma^2) par Box-Muller"""
    u1 = random.random()
    u2 = random.random()
    z = math.sqrt(-2 * math.log(u1)) * math.cos(2 * math.pi * u2)
    return mu + sigma * z

# Simuler et vérifier les propriétés
n = 100000
echantillon = [normale(mu=5, sigma=2) for _ in range(n)]
mean = sum(echantillon) / n
variance = sum((x - mean)**2 for x in echantillon) / n

print(f"Loi N(5, 4) simulée :")
print(f" Moyenne empirique : {mean:.4f} (théorique μ=5)")
print(f" Variance empirique : {variance:.4f} (théorique σ²=4)")

# Règle des 68-95-99.7
mu, sigma = 5, 2
dans_1sigma = sum(1 for x in echantillon if abs(x-mu) <= sigma) / n
dans_2sigma = sum(1 for x in echantillon if abs(x-mu) <= 2*sigma) / n
dans_3sigma = sum(1 for x in echantillon if abs(x-mu) <= 3*sigma) / n

print(f"\nRègle 68-95-99.7 :")
print(f" |X-μ| ≤ σ : {dans_1sigma*100:.1f}% (théorique 68.3%)")
print(f" |X-μ| ≤ 2σ : {dans_2sigma*100:.1f}% (théorique 95.4%)")
print(f" |X-μ| ≤ 3σ : {dans_3sigma*100:.1f}% (théorique 99.7%)")

# Densité normale (formule)
def densite_normale(x, mu=0, sigma=1):
    return (1/(sigma*math.sqrt(2*math.pi))) * math.exp(-(x-mu)**2/(2*sigma**2))

# Vérifier que l'intégrale vaut 1
total = sum(densite_normale(x/100) * 0.01 for x in range(-500, 501))
print(f"\n∫ f(x)dx ≈ {total:.4f} (doit être ≈ 1)")
```

[PIÈGE]

Pour une variable continue, $P(X = k) = 0$ pour toute valeur exacte k !
On ne peut calculer que $P(a \leq X \leq b)$ via l'intégrale de la densité.
Cela ne veut pas dire que c'est impossible, juste "de mesure nulle".

[THÉORIE]

Les statistiques descriptives RÉSUMENT et DÉCRIVENT un ensemble de données.
On s'intéresse à la tendance centrale (où sont concentrées les données)
et à la dispersion (comment elles s'étalent).

[DÉFINITION]

TENDANCE CENTRALE :

Moyenne ($\bar{x}$) : somme divisée par n
Médiane (M) : valeur centrale quand les données sont triées
Mode : valeur la plus fréquente

DISPERSION :

Variance : $s^2 = (1/n) \sum (x_i - \bar{x})^2$
Écart-type : $s = \sqrt{s^2}$
Étendue : $\max - \min$
IQR (interquartile) : $Q3 - Q1$

QUARTILES :

$Q1$ = 25^e percentile (25% des données en dessous)
 $Q2$ = Médiane = 50^e percentile
 $Q3$ = 75^e percentile

CORRÉLATION de Pearson r :

$r = \text{Cov}(X,Y) / (\sigma_X \cdot \sigma_Y) \in [-1, 1]$
 $r \approx 1 \rightarrow$ corrélation positive forte
 $r \approx -1 \rightarrow$ corrélation négative forte
 $r \approx 0 \rightarrow$ pas de corrélation linéaire

[DÉMONSTRATION] Moyenne minimise la somme des carrés des écarts

On cherche μ tel que $\sum (x_i - \mu)^2$ soit minimal.

On dérive par rapport à μ et on égale à 0 :

$$d/d\mu [\sum (x_i - \mu)^2] = \sum 2(x_i - \mu)(-1) = -2 \cdot \sum (x_i - \mu) = 0$$

$$\square \sum x_i = n \cdot \mu \quad \square \quad \mu = (1/n) \sum x_i = \bar{x} \quad \square$$

[CODE PYTHON]

```
import math

donnees = [12, 18, 24, 15, 30, 22, 19, 8, 25, 17, 21, 13, 28, 16, 20]

# Tendance centrale
def moyenne(data):
    return sum(data) / len(data)

def mediane(data):
    tri = sorted(data)
    n = len(tri)
    if n % 2 == 0:
        return (tri[n//2 - 1] + tri[n//2]) / 2
    return tri[n//2]

def mode(data):
    from collections import Counter
    c = Counter(data)
    max_freq = max(c.values())
    return [k for k, v in c.items() if v == max_freq]

print(f"Données : {donnees}")
print(f"Moyenne : {moyenne(donnees):.2f}")
print(f"Médiane : {mediane(donnees):.2f}")

# Dispersion
mu = moyenne(donnees)
variance = sum((x - mu)**2 for x in donnees) / len(donnees)
ecart_type = math.sqrt(variance)

print(f"\nVariance : {variance:.2f}")
print(f"Écart-type : {ecart_type:.2f}")
print(f"Étendue : {max(donnees) - min(donnees)}")

# Quartiles
def quartile(data, q):
    """q ∈ {0.25, 0.5, 0.75}"""
```

```

tri = sorted(data)
n = len(tri)
idx = q * (n - 1)
lo, hi = int(idx), min(int(idx) + 1, n-1)
return tri[lo] + (idx - lo) * (tri[hi] - tri[lo])

Q1 = quartile(donnees, 0.25)
Q3 = quartile(donnees, 0.75)
print(f"\nQ1 = {Q1:.1f}, Q3 = {Q3:.1f}, IQR = {Q3-Q1:.1f}")

# Corrélation Pearson
def pearson(X, Y):
    n = len(X)
    mx, my = moyenne(X), moyenne(Y)
    num = sum((X[i]-mx)*(Y[i]-my) for i in range(n))
    den = math.sqrt(sum((X[i]-mx)**2 for i in range(n)) *
                      sum((Y[i]-my)**2 for i in range(n)))
    return num / den if den != 0 else 0

X = [1, 2, 3, 4, 5]
Y = [2, 4, 5, 4, 5]
print(f"\nCorrélation Pearson X,Y : {pearson(X, Y):.4f}")

```

[PIÈGE]

Corrélation $\neq$ causalité ! Si $r = 0.95$ entre X et Y, cela NE PROUVE PAS que X cause Y. Il peut y avoir une cause commune, ou une coïncidence (spurious correlation).

22. STATISTIQUES INFÉRENTIELLES – bases

[THÉORIE]

Les statistiques inférentielles permettent de tirer des conclusions sur une POPULATION à partir d'un ÉCHANTILLON. On généralise, on teste des hypothèses, on construit des intervalles de confiance.

[DÉFINITION]

ESTIMATION :

Estimateur $\hat{\theta}$ d'un paramètre θ construit depuis l'échantillon.
 Estimateur sans biais : $E[\hat{\theta}] = \theta$
 La moyenne empirique $\bar{x}$ est un estimateur sans biais de μ .

INTERVALLE DE CONFIANCE à 95% pour μ (variance connue σ^2) :

IC = $[\bar{x} - 1.96 \cdot \sigma / \sqrt{n}, \bar{x} + 1.96 \cdot \sigma / \sqrt{n}]$
 (Si σ inconnue, on utilise la loi de Student avec s)

TEST D'HYPOTHÈSE :

H_0 : hypothèse nulle (ce qu'on cherche à réfuter)
 H_1 : hypothèse alternative
 p-valeur : probabilité d'observer les données SI H_0 était vraie
 Si p-valeur $< 0.05 \rightarrow$ on rejette H_0 au seuil 5%.

THÉORÈME CENTRAL LIMITE :

Si $X_1, \dots, X_n$ i.i.d. avec $E[X_i] = \mu$, $\text{Var}(X_i) = \sigma^2$:
 $\sqrt{n} \cdot (\bar{x} - \mu) / \sigma \rightarrow N(0,1)$ quand $n \rightarrow \infty$

[DÉMONSTRATION partielle] Le TCL explique pourquoi la normale est partout

Quand on additionne beaucoup de variables aléatoires indépendantes (même si elles ne sont pas normales), leur somme normalisée converge vers une distribution normale. C'est pourquoi tant de mesures physiques, biologiques et sociales suivent une loi normale : elles sont la somme de nombreux petits effets indépendants. $\square$

[CODE PYTHON]

```

import math
import random

# Théorème Central Limite : démonstration visuelle
def tirer_somme(n_variannes, n_simulations=10000):
    """Simule la somme de n variables uniformes U([0,1])"""
    sommes = []

```



```

for _ in range(n_simulations):
    s = sum(random.random() for _ in range(n_variables))
    sommes.append(s)
return sommes

print("Moyenne et variance de sommes de variables uniformes U([0,1]) :")
for n in [1, 5, 20, 100]:
    sommes = tirer_somme(n)
    mean = sum(sommes) / len(sommes)
    var = sum((x - mean)**2 for x in sommes) / len(sommes)
    print(f"  n={n:>3} : moyenne={mean:.4f} (théo={n/2:.4f}), "
          f"var={var:.4f} (théo={n/12:.4f})")

# Intervalle de confiance à 95%
def IC_95(echantillon, sigma_connu=None):
    """
    Intervalle de confiance à 95% pour la moyenne.
    Si sigma_connu est fourni, utilise z=1.96.
    Sinon, utilise l'écart-type empirique (approximation).
    """
    n = len(echantillon)
    mean = sum(echantillon) / n

    if sigma_connu is not None:
        sigma = sigma_connu
    else:
        sigma = math.sqrt(sum((x-mean)**2 for x in echantillon) / n)

    marge = 1.96 * sigma / math.sqrt(n)
    return mean - marge, mean + marge

# Simulation : tirer 30 mesures d'une N(10, 4), calculer IC
echantillon = [10 + 2*(random.gauss(0,1)) for _ in range(30)]
lo, hi = IC_95(echantillon)
mean = sum(echantillon)/len(echantillon)
print(f"\nÉchantillon de 30 valeurs (μ_réel=10, σ=2) :")
print(f"  x̄ = {mean:.3f}")
print(f"  IC 95% : [{lo:.3f}, {hi:.3f}]")
print(f"  Contient μ=10 ? {'✓ Oui' if lo <= 10 <= hi else 'x Non'}")

```

[PIÈGE]

Un IC à 95% ne signifie PAS "il y a 95% de chances que μ soit dans cet intervalle".
 μ est un PARAMÈTRE FIXE, pas aléatoire. La bonne interprétation :
 "95% des intervalles construits ainsi contiendraient le vrai μ ".

BLOC F – MATHÉMATIQUES AVANCÉES

23. NOMBRES COMPLEXES – applications

[THÉORIE]

Les nombres complexes $\mathbb{C} = \{a + bi \mid a, b \in \mathbb{R}\}$ permettent de résoudre toute équation polynomiale (théorème fondamental de l'algèbre).
 En ingénierie, ils modélisent les signaux oscillants et les circuits électriques.
 La formule d'Euler les lie à la trigonométrie.

[DÉFINITION]

$z = a + bi$ où $i^2 = -1$
 Module : $|z| = \sqrt{a^2 + b^2}$
 Argument : $\arg(z) = \text{angle } \theta$ tel que $a = |z|\cos\theta$, $b = |z|\sin\theta$
 Conjugué : $z^- = a - bi$

FORME POLAIRE (EULER) : $z = |z| \cdot e^{(i\theta)} = |z|(\cos\theta + i \cdot \sin\theta)$

FORMULE D'EULER : $e^{(i\pi)} + 1 = 0$ (la plus belle formule des maths !)

OPÉRATIONS EN POLAIRE :

$z_1 \cdot z_2$: modules se multiplient, arguments s'additionnent
 z^n : module élevé à n , argument multiplié par n (formule de Moivre)

[DÉMONSTRATION] $e^{i\theta} = \cos \theta + i \cdot \sin \theta$ (via développements en série)

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

$$\cos \theta = 1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \dots$$

$$\sin \theta = \theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \dots$$

$$e^{i\theta} = 1 + i\theta + \frac{(i\theta)^2}{2!} + \frac{(i\theta)^3}{3!} + \dots$$

$$= 1 + i\theta - \frac{\theta^2}{2!} - \frac{i\theta^3}{3!} + \frac{\theta^4}{4!} + \dots$$

Parties réelles : $1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \dots = \cos \theta$

Parties imaginaires : $\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \dots = \sin \theta$

Donc $e^{i\theta} = \cos \theta + i \cdot \sin \theta$ □

[CODE PYTHON]

```
import cmath
import math

# Représentation complexe
z1 = 3 + 4j
z2 = 1 - 2j

print(f"z1 = {z1}")
print(f"Module |z1| = {abs(z1)}") # 5.0
print(f"Argument arg(z1) = {cmath.phase(z1):.4f} rad = "
      f"{math.degrees(cmath.phase(z1)):.1f}°")
print(f"Conjugué z1* = {z1.conjugate()}")

# Opérations
print(f"\nz1 + z2 = {z1 + z2}")
print(f"z1 * z2 = {z1 * z2}")
print(f"z1 / z2 = {z1 / z2:.4f}")

# Formule d'Euler : e^(in) + 1 = 0
euler = cmath.exp(1j * math.pi)
print(f"\ne^(in) = {euler.real:.10f} + {euler.imag:.10f}i")
print(f"e^(in) + 1 = {(euler + 1).real:.2e} (≈ 0 ✓)")

# Formule de Moivre : (cosθ + i·sinθ)^n = cos(nθ) + i·sin(nθ)
theta = math.pi / 6
n = 5
z = math.cos(theta) + 1j * math.sin(theta)
print(f"\nFormule de Moivre :")
print(f"z^n par Moivre : {(math.cos(n*theta)) + 1j*(math.sin(n*theta)):.4f}")
print(f"z^n direct : {z**n:.4f}") # Mêmes résultats ✓

# Racines nièmes de l'unité
n = 6
racines = [cmath.exp(2j * math.pi * k / n) for k in range(n)]
print(f"\nRacines 6ièmes de 1 :")
for k, r in enumerate(racines):
    print(f" k={k}: {round(r.real,4):>8} + {round(r.imag,4):>8}i")
```

[PIÈGE]

$\sqrt{-1} = i$ en maths... mais en Python, `math.sqrt(-1)` génère une erreur.
Il faut utiliser `cmath.sqrt(-1)` ou écrire `1j` directement.

24. THÉORIE DES GRAPHS – bases

[THÉORIE]

Un graphe modélise des RELATIONS entre objets : réseaux sociaux, routes, circuits, molécules, internet... C'est une abstraction mathématique incroyablement puissante. Les algorithmes de graphes sont au cœur de Google, GPS, recommandation, etc.

[DÉFINITION]

GRAPHE $G = (V, E)$ où V = sommets (vertices), E = arêtes (edges)
 DEGRÉ $d(v)$: nombre de voisins de v
 CHEMIN : suite de sommets reliés par des arêtes
 CYCLE : chemin qui revient à son départ

GRAPHE ORIENTÉ : les arêtes ont un sens ($A \rightarrow B \neq B \rightarrow A$)

REPRÉSENTATIONS :

Matrice d'adjacence : $A_{i,j} = 1$ si arête (i,j) , 0 sinon
Liste d'adjacence : pour chaque sommet, la liste de ses voisins

PROPRIÉTÉS :

CONNEXE : il existe un chemin entre toute paire de sommets
ARBRE : graphe connexe sans cycle (n sommets, $n-1$ arêtes)
EULÉRIEN : graphe admettant un circuit qui passe par chaque arête une fois

ALGORITHMES CLASSIQUES :

BFS (parcours en largeur) : trouver le plus court chemin non pondéré
DFS (parcours en profondeur) : détecter les cycles
Dijkstra : plus court chemin dans un graphe pondéré positif

[DÉMONSTRATION] Théorème de la poignée de mains

Dans tout graphe, $\sum d(v) = 2|E|$ (somme des degrés = $2 \times$ nombre d'arêtes)
Car chaque arête contribue 1 au degré de chacune de ses deux extrémités.
Corollaire : le nombre de sommets de degré impair est toujours PAIR. $\square$

[CODE PYTHON]

```
from collections import deque, defaultdict

class Graphe:
    def __init__(self, oriente=False):
        self.adj = defaultdict(set)
        self.oriente = oriente

    def ajouter_arete(self, u, v):
        self.adj[u].add(v)
        if not self.oriente:
            self.adj[v].add(u)

    def sommets(self):
        return set(self.adj.keys())

    def degres(self):
        return {v: len(voisins) for v, voisins in self.adj.items()}

    def bfs(self, depart, arrivee):
        """Plus court chemin par BFS"""
        if depart == arrivee:
            return [depart]
        visites = {depart}
        file = deque([(depart, [depart])])
        while file:
            sommet, chemin = file.popleft()
            for voisin in self.adj[sommet]:
                if voisin == arrivee:
                    return chemin + [voisin]
                if voisin not in visites:
                    visites.add(voisin)
                    file.append((voisin, chemin + [voisin]))
        return None # Pas de chemin

    def est_connexe(self):
        if not self.sommets():
            return True
        depart = next(iter(self.sommets()))
        visites = set()
        file = deque([depart])
        while file:
            s = file.popleft()
            if s not in visites:
                visites.add(s)
                file.extend(self.adj[s] - visites)
        return visites == self.sommets()

# Exemple
G = Graphe()
for u, v in [('A','B'), ('A','C'), ('B','D'), ('C','D'), ('D','E')]:
    G.ajouter_arete(u, v)

print("Degrés :", G.degres())
print("Connexe :", G.est_connexe())
```

```
print("Chemin A-E :", G.bfs('A', 'E'))

# Vérifier théorème poignée de mains
degres = G.degres()
somme_degres = sum(degres.values())
nb_aretes = sum(len(v) for v in G.adj.values()) // 2
print(f"\nΣ degrés = {somme_degres} = 2 × {nb_aretes} arêtes ✓")
```

[PIÈGE]

Un graphe CONNEXE ≠ un graphe COMPLET.
 Connexe : on peut aller de partout à partout (via des chemins).
 Complet : il y a une arête DIRECTE entre chaque paire de sommets.
 Tout graphe complet est connexe, mais l'inverse est faux.

25. LOGIQUE & DÉMONSTRATIONS – synthèse et méta-mathématiques

[THÉORIE]

Ce dernier domaine clôt le cours en revenant à ses fondements : les MÉTHODES DE PREUVE. Savoir calculer c'est bien ; savoir DÉMONTRER c'est mieux. Toutes les mathématiques reposent sur ces quelques stratégies. Le mathématicien choisit sa stratégie de démonstration comme un programmeur choisit son algorithme.

[DÉFINITION]

5 STRATÉGIES DE DÉMONSTRATION :

1. PREUVE DIRECTE :
 $P \rightarrow Q$: on suppose P , on raisonne étape par étape jusqu'à conclure Q .
 Quand l'utiliser : la chaîne causale est claire.
2. PREUVE PAR CONTRAPOSÉE :
 $P \rightarrow Q$ est équivalent à $(\neg Q \rightarrow \neg P)$.
 Quand l'utiliser : $\neg Q \rightarrow \neg P$ est plus simple à montrer.
3. PREUVE PAR L'ABSURDE :
 Supposer $\neg P$, raisonner jusqu'à une contradiction $\rightarrow P$ est vrai.
 Quand l'utiliser : $\neg P$ mène à une impasse évidente.
4. PREUVE PAR RÉCURRENCE (induction) :
 Pour $\forall n \in \mathbb{N}$, $P(n)$:
 - Base : $P(0)$ vraie
 - Hérité : $P(n) \rightarrow P(n+1)$
 Quand l'utiliser : P dépend d'un entier n .
5. PREUVE PAR CONTRE-EXEMPLE :
 Pour réfuter $\forall x, P(x)$: trouver un x_0 tel que $\neg P(x_0)$.
 Quand l'utiliser : réfuter une affirmation universelle.

[DÉMONSTRATION – les 5 stratégies sur des exemples]

DIRECTE : "Si n est pair, alors n^2 est pair"

Supposons n pair. Alors $\exists k, n = 2k$.

$$n^2 = (2k)^2 = 4k^2 = 2(2k^2) \rightarrow n^2 \text{ est pair. } \square$$

CONTRAPOSÉE : "Si n^2 est impair, alors n est impair"

Contraposée : "Si n est pair, alors n^2 est pair" (prouvée ci-dessus). $\square$

ABSURDE : " $\sqrt{2}$ est irrationnel"

Supposons $\sqrt{2} = p/q$ avec $\text{pgcd}(p,q)=1$ (fraction irréductible).

$$2 = p^2/q^2 \rightarrow p^2 = 2q^2 \rightarrow p^2 \text{ pair} \rightarrow p \text{ pair} \rightarrow p = 2m$$

$$\rightarrow 4m^2 = 2q^2 \rightarrow q^2 = 2m^2 \rightarrow q \text{ pair.}$$

Contradiction : p et q tous deux pairs, mais $\text{pgcd}(p,q) = 1$.

Donc $\sqrt{2}$ est irrationnel. $\square$

RÉCURRENCE : $\sum_{k=1}^n k = n(n+1)/2$

$$\text{Base } n=1 : 1 = 1(2)/2 = 1 \checkmark$$

Hérité : supposons vrai au rang n .

$$\sum_{k=1}^{n+1} k = [n(n+1)/2] + (n+1) = (n+1)(n/2 + 1) = (n+1)(n+2)/2$$

C'est bien la formule au rang $n+1$. $\square$

CONTRE-EXEMPLE : "Tout entier p tel que $2^p - 1$ est premier est premier"

$p = 11 : 2^{11}-1 = 2047 = 23 \times 89$ (pas premier !)
Donc l'assertion est fausse. $\square$

[CODE PYTHON – Vérificateur de stratégies]

Un mini-framework pour explorer et tester les démonstrations

```
def verifier_implication(P, Q, domaine, nom="P⇒Q"):
    """
    Vérifie P⇒Q sur un domaine fini.
    Retourne les contre-exemples si l'implication est fausse.
    """
    contre_exemples = [x for x in domaine if P(x) and not Q(x)]
    if contre_exemples:
        print(f"x {nom} : FAUSSE. Contre-exemples : {contre_exemples[:5]}")
    else:
        print(f"✓ {nom} : semble vraie sur ce domaine (pas de preuve formelle !)")
    return len(contre_exemples) == 0

# Stratégie directe : "n pair ⇒ n² pair"
entiers = list(range(-50, 51))
verifier_implication(
    lambda n: n % 2 == 0,
    lambda n: (n**2) % 2 == 0,
    entiers,
    "n pair ⇒ n² pair"
)

# Contre-exemple : "n² pair ⇒ n pair" (vraie en réalité !)
verifier_implication(
    lambda n: (n**2) % 2 == 0,
    lambda n: n % 2 == 0,
    entiers,
    "n² pair ⇒ n pair"
)

# Trouver les contre-exemples pour une fausse assertion
def trouver_contre_exemple(predicat, domaine):
    """Cherche un x dans le domaine qui réfute le prédicat"""
    for x in domaine:
        if not predicat(x):
            return x
    return None

# "Tout nombre de la forme n² + n + 41 est premier" (formule de Euler)
def est_premier(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False
    return True

formule_euler = lambda n: est_premier(n**2 + n + 41)
domaine = list(range(0, 100))
ce = trouver_contre_exemple(formule_euler, domaine)
print(f"\nContre-exemple à 'n²+n+41 toujours premier' :")
print(f"  n = {ce} : {ce}²+{ce}+41 = {ce**2+ce+41} (premier? {est_premier(ce**2+ce+41)})")
# n=40 : 40²+40+41 = 1681 = 41² (pas premier !)

# Vérificateur de récurrence (n premiers rangs)
def verifier_recurrence(base, heredite, n_max=100):
    """
    Vérifie que P(0) est vraie et que P(n)⇒P(n+1) tient
    sur les n_max premiers rangs.
    """
    if not base(0):
        print(f"x Base P(0) fausse !")
        return False
    for n in range(n_max):
        if base(n) and not base(n+1):
            print(f"x Hérité échoue en n={n}")
            return False
    print(f"✓ Récurrence vérifiée sur [0, {n_max}]")
    return True

# Σ(k=1 to n) k = n(n+1)/2
```

```
somme_entiers = lambda n: n*(n+1)//2 == sum(range(n+1))
verifier_reccurrence(somme_entiers, None, n_max=200)
```

[PIÈGE]

Vérifier une propriété sur 1000 exemples NE PROUVE PAS qu'elle est toujours vraie.
 La "formule d'Euler" n^2+n+41 fonctionne pour $n=0..39$ mais échoue en $n=40$.
 En mathématiques, la PREUVE formelle est la seule garantie absolue.

RÉCAPITULATIF FINAL – LES 25 DOMAINES

BLOC A – ALGÈBRE & ARITHMÉTIQUE

1	Algèbre	Équations, polynômes, discriminant
2	Systèmes d'équations	Gauss, matrices, rang
3	Arithmétique modulaire	Congruences, Fermat, cryptographie
4	Combinatoire	Arrangements, $C(n,k)$, binôme

BLOC B – FONCTIONS & ANALYSE

5	Fonctions réelles	Parité, monotonie, périodicité
6	Limites & continuité	Définition ε - δ , L'Hôpital, formes ∞
7	Dérivation	Pente, extremums, règles de calcul
8	Intégration	Aire, primitives, par parties
9	Suites numériques	Arithm., géom., convergence, Fib.

BLOC C – TRIGONOMÉTRIE & GÉOMÉTRIE

10	Trigonométrie fondamentale	Cercle trigo., sin/cos/tan, radians
11	Trigonométrie avancée	Formules d'addition, équations
12	Géométrie analytique	Droites, cercles, distance
13	Géométrie vectorielle	Norme, prod. scalaire, angles

BLOC D – ALGÈBRE LINÉAIRE

14	Espaces vectoriels	Base, dimension, indépendance
15	Matrices	Opérations, transposée, valeurs prop
16	Déterminants & systèmes	det, inversibilité, Cramer

BLOC E – PROBABILITÉS & STATISTIQUES

17	Probabilités fondamentales	Axiomes Kolmogorov, inclusion-excl.
18	Prob. conditionnelles/Bayes	$P(A B)$, indépendance, base rate
19	V.A. discrètes	Bernoulli, binomiale, Poisson
20	V.A. continues	Normale, uniforme, densité
21	Statistiques descriptives	Moyenne, médiane, variance, IQR
22	Statistiques inférentielles	IC, tests, TCL, p-valeur

BLOC F – MATHÉMATIQUES AVANCÉES

23	Nombres complexes	Euler, forme polaire, Moivre
24	Théorie des graphes	BFS/DFS, connexité, arbres
25	Logique & démonstrations	5 stratégies, récurrence, absurde

#####

```
#
# CE QUE TU MAÎTRISES MAINTENANT – RÉCAPITULATIF COMPLET DES DEUX CHAPITRES
#
# ✓ RAISONNEMENT – Propositions, connecteurs, quantificateurs, preuves
# ✓ ALGÈBRE – Équations, polynômes, systèmes, congruences, combina.
# ✓ ANALYSE – Fonctions, limites, dérivées, intégrales, suites
# ✓ TRIGONOMÉTRIE – Cercle trig., formules d'addition, équations trig.
# ✓ GÉOMÉTRIE – Analytique, vectorielle, distances, angles
# ✓ ALGÈBRE LIN. – Vecteurs, matrices, déterminants, valeurs propres
# ✓ PROBABILITÉS – Axiomes, Bayes, V.A. discrètes et continues
# ✓ STATISTIQUES – Descriptives, inférentielles, TCL, IC
```

```
# ✓ NOMBRES          -  $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$  + arithmétique modulaire
# ✓ GRAPHES          - Sommets, arêtes, BFS, connexité, arbres
#
# PROCHAINES ÉTAPES NATURELLES :
#   → Analyse avancée (séries de Taylor, équations différentielles)
#   → Algèbre abstraite (groupes, anneaux, corps)
#   → Topologie (espaces métriques, compacité)
#   → Théorie des probabilités avancée (martingales, processus stochastiques)
#   → Mathématiques appliquées (optimisation, ML, cryptographie)
#
#####
```